SxSSD: A Secure and Extensible Software-defined Solid State Drive

Josh Dafoe
Department of Computer Science
Michigan Technological University
Houghton, Michigan, USA
jwdafoe@mtu.edu

Bo Chen
Department of Computer Science
Michigan Technological University
Houghton, Michigan, USA
bchen@mtu.edu

Abstract

Solid-state drives (SSDs) are built on NAND flash memory and typically expose it to the operating system through a block-based storage interface. As NAND flash has special read/write constraints to accommodate its unique hardware nature, a translation between OS-level I/Os and raw flash memory I/Os is needed. This results in a flash translation layer (FTL) that essentially creates a "trusted computing base" due to its physical isolation from the OS. Building on this trusted computing base, some security designs (e.g., secure data recovery from malware attacks) have been proposed that can ensure strong data security properties even if the OS is compromised. However, they mostly require directly modifying the FTL's firmware code, which is hard to achieve in practice because the traditional block-based FTL does not provide an interface to modify its internal functions. New flash storage interface designs, such as open-channel SSDs or zoned namespaces, have moved key FTL functions into the OS. These new interfaces alleviate the difficulty of modifying FTL functions, at the cost of blurring the trusted boundary, as the FTL is no longer isolated from the OS.

In this work, we have introduced SxSSD, the secure yet extensible software-defined SSD design. By decoupling the internal policy definitions from the primitive FTL mechanisms, we allow trusted applications to dynamically and securely define the FTL policies and the exposed storage interface (achieving increased flexibility compared to the open-channel and zoned namespaces SSDs). Most significantly, SxSSD can retain the isolation of traditional FTL execution (achieving a level of security similar to that of traditional block-based SSDs). We have identified and addressed a few key security challenges introduced under a compromised OS. In addition, we have implemented a prototype of SxSSD and evaluated its overhead with different FTL policies and storage interfaces. The experimental evaluation demonstrates that the overhead incurred by SxSSD is small compared to native FTL implementations.

Keywords Solid-state Drives · Flash Translation Layer · Extensibility · ZNS · Open-channel SSDs

1 Introduction

Solid-state drives (SSDs), a widely used form of data storage, are largely based on NAND flash memory. Compared to traditional magnetic storage (e.g. hard disk drives), NAND flash exhibits a few special hardware characteristics, which enforce unique low-level storage semantics. Specifically, data stored on NAND flash must be erased in block-sized units, typically a few hundred KiBs, while reads and writes occur in page-sized units, typically a few KiBs. In addition, updating a page requires erasing its entire containing block first, which means that data should not be overwritten in place. Therefore, a simple write request issued by the user application requires a complicated decision based on global metadata maintained throughout the flash. To give the operating system (OS) a consistent storage interface similar to traditional magnetic storage while also addressing the unique hardware constraints of flash memory, a *translation* from OS-level storage requests to actual flash memory operations is necessary.

To enable this translation, a special piece of firmware, called a flash translation layer (FTL), has been introduced into modern SSDs. The FTL provides the host operating system with a logical interface (we call it *flash-to-OS interface*, as

shown in Figure 1) that differs from the raw flash memory interface, translating the I/O requests issued by the OS into sequences of operations supported by the underlying NAND flash. For example, a *block-interface FTL* has been broadly implemented in mainstream SSDs. The OS, in turn, often provides applications with another storage interface (we call it *OS-to-application interface*, as shown in Figure 1) that is different from the one provided by the FTL. Therefore, the storage semantics ultimately observed by the applications are determined by both the flash-to-OS and OS-to-application interfaces.

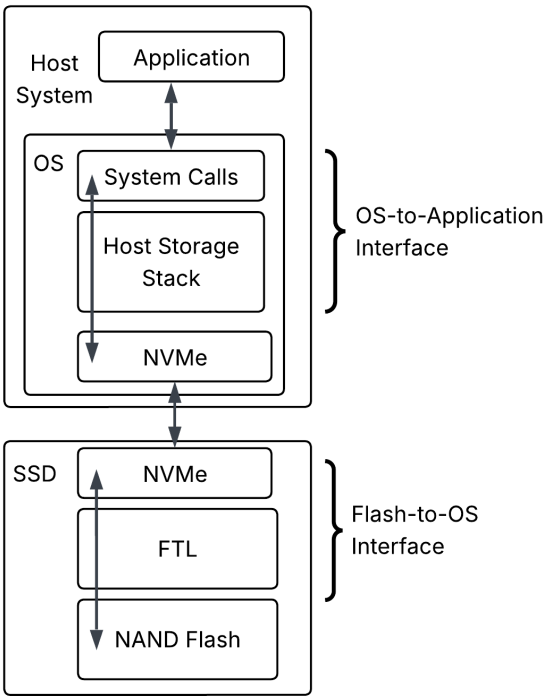

Figure 1: An overview of the host and device storage stack, assuming an NVMe SSD is used. The OS-to-application interface is implemented in the host storage stack, while the flash-to-OS interface is implemented in the device-side FTL.

Previous work has introduced software-defined SSDs (e.g., zoned namespaces (ZNS), open-channel SSD (OCSSD)) by exposing flash-to-OS interfaces that are less constrained than the traditional block interface and, therefore, closer to the underlying flash operations [1, 2, 3, 4, 5, 6]. This gives the host greater control over data placement and functions traditionally handled within the FTL, allowing the operating system to construct custom OS-to-application abstractions tailored to different workloads. As a result, applications can be co-designed with the storage stack [7, 8], and different storage semantics can be exposed to different applications [6].

Our key observation is that all prior SSD designs (including block-interface SSDs and software-defined SSDs such as ZNS and OCSSD) share a fundamental limitation: *they expose a fixed flash-to-OS interface*. This limitation leads to two negative consequences that prevent existing FTL designs from meeting the diverse requirements of emerging applications:

(1) Difficulty in Building High-performance Storage Systems. Previous work has consistently demonstrated that flash storage performance depends strongly on the dynamic interaction between specific workloads and the underlying flash management strategies [9, 10, 4]. However, implementing this "smart" interaction is hard in existing FTL designs, as they all expose a fixed flash-to-OS interface. For block-interface SSDs, the OS typically interacts with the FTL via a limited block-access interface, making it difficult to coordinate the application workloads with the FTL. For example, many applications may suffer from the log on log problem [11], resulting in unnecessary duplicate functions between FTL and the upper-layer software [11]. Software-defined SSDs such as ZNS and OCSSD may partially address this problem by introducing programmability of the OS-to-application interface. However, since their flash-to-OS interface is still fixed, they remain constrained by a single set of storage semantics that inherently favors some workloads over others. For example, ZNS can make some host-side operations (e.g. zone compaction) more expensive than necessary by forcing them through a limited flash-to-OS interface [12, 13]. Overall, although many applications obtain

performance benefits from modifying the device-level FTL [13, 14, 15, 16, 17, 18], the fixed flash-to-OS interface makes such modifications difficult in practice.

(2) Difficulty in supporting critical security storage semantics. The device-side FTL is isolated from the host OS by SSD hardware and is therefore a critical security boundary when facing a compromised OS [19, 20]. Previous work has leveraged this by designing modified block-interface FTLs to provide security-critical storage semantics to trusted applications, such as plausibly deniable storage [21], secure data recovery from malware attacks [22, 20, 19, 23], integrity-assured cloud storage [24], etc. Since these modifications rely on the trustworthy nature of the device-side FTL, implementing these designs currently requires explicit modifications to the FTL firmware. Thus, it is difficult (or even impossible) for existing SSDs (including both block-interface and software-defined SSDs) to support such security-critical designs due to the inflexible nature of the flash-to-OS interface in existing FTLs. Notably, the existing software-defined SSDs do not address this problem: While they introduce custom storage semantics at the OS-to-application interface, they fail to support the class of security-critical storage semantics because the flash-to-OS interface remains fixed.

Until now, there has been no existing approach that can overcome the fundamental limitation mentioned above. In this work, we introduce SxSSD, the **S**ecure yet e**x**tensible software-defined **S**olid **S**tate **D**rive, aiming to close this gap. Instead of introducing another FTL with a fixed flash-to-OS interface, SxSSD proposes to introduce extensibility on top of a set of generic FTL procedures by allowing the flash-to-OS storage interface and the supporting FTL policies to be dynamically defined while maintaining the trusted device boundary. Our design is guided by two central requirements: extensibility and secure policy management:

Extensibility. We address the fundamental limitation of a fixed flash-to-OS interface through extensibility, which allows trusted applications to define new storage semantics and FTL behavior. Specifically, SxSSD achieves this extensibility through programmability, by allowing custom policy code and storage semantics to be defined and installed by trusted applications without modifying the FTL firmware. This addresses the difficulty in building high-performance storage systems by allowing FTL behavior to be dynamically tailored to specific workloads, rather than being constrained by a fixed interface. Additionally, this extensibility partially addresses the difficulty of supporting security-critical storage semantics, since such semantics are impractical to realize within the trusted device boundary without a dynamically definable flash-to-OS interface. To enable extensibility, the *first challenge* is to establish a general programming model to define custom flash-to-OS storage semantics. To address this, we characterize the FTL design space and identify a common structure in which fundamental FTL mechanisms are clearly distinguished from the policies that determine flash-to-OS storage semantics (Section 4). The *second challenge* is to provide a concrete design through which this programmability can be realized. To address this, the design of SxSSD is informed by the established mechanism-policy decomposition. Specifically, SxSSD is structured so that the core flash management mechanisms are fixed. Then, on top of these, SxSSD exposes a well-defined API (to FTL policies) that allows interaction with the underlying storage hardware, through which they realize host-visible storage semantics. To ensure that custom policies are enforced on-device, policies are (1) stored in flash storage, (2) dynamically loaded into memory, and (3) invoked through an event-driven policy engine. Finally, to expose the programmability externally, SxSSD introduces an interface for installing and managing custom device-enforced policy code. We call this new interface a *meta-interface*, since it operates at a higher level of abstraction, enabling the definition of custom storage interfaces (i.e. it is "an interface for defining interfaces"), rather than providing fixed storage semantics. Unfortunately, expanding the FTL interface in this way introduces a new attack surface through which an untrusted host OS may attempt to manipulate policies. Our next requirement effectively removes this attack surface by restricting access to the meta-interface.

Secure Policy Management. This requirement enables security-critical semantics by ensuring that policies can only be managed by trusted entities, even with an untrusted OS as a mediator. The *first challenge* is to prevent untrusted entities from modifying functionality beneath the trusted device boundary. To address this, SxSSD makes the meta-interface the exclusive mechanism through which policies can be managed. In addition, we restrict its use to trusted entities through authentication mechanisms. The *second challenge* is to ensure that the policy integrity is maintained, and the on-device policy state can be audited. To address this, SxSSD supports policy attestation and protects policy integrity by storing policies in a reserved on-device region outside the policy-visible address space. When required by the trusted application, SxSSD also protects the confidentiality of the policy during management and storage, preventing the adversarial host from inferring the semantics of the policy.

Contributions: Our contributions are as follows:

(i) We survey existing storage interface designs and characterize the FTL design space under a set of unifying abstractions.

(ii) We propose SxSSD, which restructures the FTL according to this characterization and exposes a meta-interface for dynamically defining host-visible storage semantics and installing corresponding on-device policies.

(iii) We introduce mechanisms to make extensibility secure under an untrusted host OS, including authorized policy management, policy attestation, policy freshness, policy integrity, and policy confidentiality.

(iv) We have implemented a prototype of SxSSD in an SSD emulator [25] and evaluated it on multiple real-world workloads, showing only 3.74% overhead over a baseline SSD.

(v) We conduct a case study in which we enable secure data recovery from ransomware attacks, demonstrating SxSSD can support various applications without the need to modify the FTL firmware code.

2 Background

2.1 NAND Flash Characteristics

NAND flash presents hardware characteristics that differ from magnetic storage. It uses floating gate transistors, which are organized onto a number of storage chips. These are then wired to a processor via independent channels, allowing parallelism across these channels. The storage chips are organized into a hierarchy of die, plane, block, and page. While there are multiple dies per channel, each die allows only a single flash command to be executed at a time. However, that command can be executed synchronously at aligned addresses on multiple planes in parallel. Within each plane, NAND is organized into blocks and pages; each plane contains the same number of blocks, and each block contains the same number of pages. Importantly, the parallelism exhibited by flash memory can significantly increase its performance. Therefore, the optimal strategy is to stripe the writes across the various units of parallelism (Figure 2), rather than writing sequentially within a single unit. Finally, each physical page includes a small out-of-band (OOB) area that is used internally by the FTL.

Due to the nature of floating gate transistors, there are a few constraints on the primitive read/write/erase operations: (**C1**) Writes are performed at page granularity. (**C2**) Erase operations (which are an order of magnitude more expensive than write) must be performed before overwriting a page. Additionally, primitive erase operations are performed only at the block granularity. (**C3**) Writes are often constrained to proceed sequentially within a NAND flash block. (**C4**) Flash blocks can support only a limited number of erasures before failure and therefore have a limited lifetime [26].

Together, these constraints characterize the raw storage interface exposed by NAND flash hardware. However, C1-C4 result in complex dependencies between flash operations over time, making it challenging to present a simple unified interface that effectively virtualizes storage resources among applications. Instead, I/O requests issued by higher software layers must be translated into sequences of flash operations that satisfy C1-C4 while preserving the expected storage semantics. Although such translation may occur at multiple layers in the storage stack, in this paper, we use the term *flash translation layer* (FTL) to refer specifically to the translation software running on the storage controller.

Typically, flash based SSDs are equipped with DRAM sufficient for a read/write cache and efficient FTL execution. Often, this DRAM can resist power loss failure by incorporating capacitors [27] or batteries [28].

2.2 The NVMe Interface

Mainstream SSD interface types include Serial ATA (SATA), Serial Attached SCSI (SAS), and NVM Express (NVMe), among which NVMe SSDs are expected to have the fastest average yearly growth rate [29]. For NVMe SSDs, communication between the host OS and the SSDs typically takes place using the NVMe over PCIe protocol (Figure 1). This interface relies on submission queues and completion queues resident on the host DRAM. The host invokes storage operations by placing NVMe commands into a submission queue, then the FTL fetches these commands, processes them, and reports this via a completion queue. Two aspects of the NVMe interface are particularly relevant to our design: First, NVMe simply presents the device address space as a flat logical block addressing scheme, effectively decoupling the host-side view from the internal geometry managed by the FTL. This implies that some translation into the physical address space is inherent to all FTLs. Second, the NVMe command set supports vendor-specific commands in addition to a set of standard operations (e.g., read, write). We leverage and expand this flexibility to define our own NVMe commands and enable policy-specific NVMe commands.

2.3 Case Studies of Flash Translation Layers

In the following, we survey the main FTL designs, in order to establish a unifying characterization of the FTL design space, which informs our subsequent conceptual separation between FTL mechanisms and policy (Section 4).

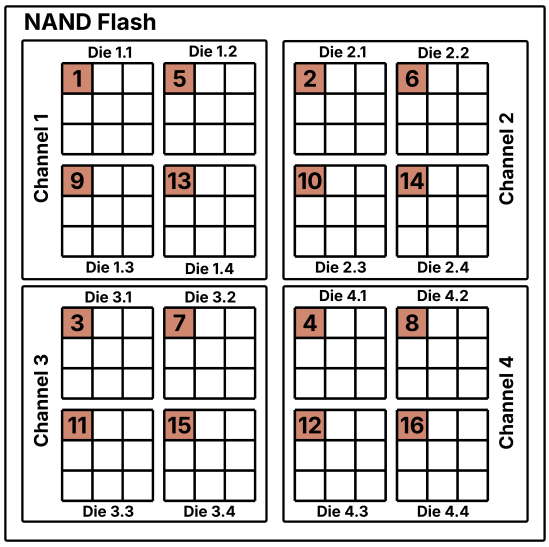

Figure 2: A simplified view of NAND flash geometry. The orange blocks make up a superblock, commonly used by block-interface and ZNS FTLs. The numbers indicate the striping order that maximizes parallelism across the superblock.

2.3.1 Open Channel SSDs

An open-channel SSD (**OCSSD**) exposes internal flash structure and parallelism to the host through its FTL, presenting the OS with an interface closely matched to raw flash semantics. Specifically, the OCSSD presents a logical address space with a fixed mapping onto the device geometry that reflects hardware parallelism. As a consequence, the OS can affect performance via direct control over data placement.

Complying with C1, an OCSSD defines the smallest addressable unit for read/write as a logical block, often corresponding to a physical page. Logical blocks are organized into chunks, each associated with a parallel unit (PU), with PUs organized into groups. Often, groups correspond to channels, and PUs correspond to dies. Writes must advance sequentially within chunks along an FTL-maintained write pointer. To exploit parallelism within a chunk, the write pointer advances at the same page index across different planes. Then, C3 is addressed by advancing the write pointer to the next page index after exhausting the planes. This implies that each chunk consists of a fixed set of aligned physical blocks within its associated PU, so each physical block belongs to exactly one chunk. In addition to a write pointer, the OCSSD FTL maintains chunk state information (free, open, closed, or offline). To address C2, the enforced erase unit is a chunk (i.e., a fixed set of aligned physical blocks within a PU), which must be reset before it can be rewritten. In response to C4, the OCSSD FTL tracks the wear status of the chunk and reports this to the host, which is responsible for implementing wear leveling policies to evenly distribute wear between chunks. Malfunctioning chunks are retired in the FTL by internally setting them to the offline state [2].

2.3.2 Zoned Namespaces SSDs

Rather than exposing the raw flash geometry and device parallelism to the host, the zoned namespaces (**ZNS**) interface simply exposes logical zones with a sequential write constraint. Although zones are fixed logical regions of the namespace, their placement on physical media is managed internally by the ZNS FTL and may change over time. Although implementation details can vary, zones typically span aligned block addresses across multiple dies, with the write pointer advancing along aligned page offsets in those blocks. A zone may even cover an entire superblock, which is a common block address that spans all channels, dies, and planes (Figure 2). Constraints C1 and C3 are addressed similarly to the OCSSD. Specifically, logical blocks are the smallest unit of read/write, and sequential writes within zones ensure that the write pointer advances without violating C3. Additionally, the ZNS FTL maintains a per-zone write pointer and tracks zone state (empty, open, closed, full, read-only, or offline). To address C2, the enforced unit of erase is a zone, which must be reset before it can be rewritten. Unlike OCSSD, C4 is addressed by performing wear-leveling in the FTL, in which physical resources are dynamically allocated to zones to spread erase counts evenly across blocks [30]. In addition, zones are retired after failures by moving them into the offline state. Importantly, the ZNS SSD is considered a successor to OCSSD [4, 5], and can be implemented on top of the OCSSD interface [7].

2.3.3 Block-interface SSDs

Block-interface FTLs hide the flash storage constraints from the host. This is achieved by presenting a standard logical block interface, enabling random read/write operations over the entire logical address space. For C1, the unit of read/write is maintained as a logical page, corresponding to the size of a physical page. To handle C2, the FTL implements out-of-place updates, where logical overwrites are written at different physical locations. Typically, the new writes are striped along superblocks (Figure 2), to maximize internal parallelism (thereby satisfying C3 similarly to OCSSD and ZNS). When this occurs, the old physical locations are invalidated. Invalid pages accumulate over time and must be reclaimed by on-device garbage collection (typically at the superblock level). In addition, to keep track of the evolving physical location of a given logical page, a mapping table is maintained.

To address C4, the block-interface FTL performs internal wear leveling to balance erase counts across blocks. Usually, this is done by migrating blocks (or superblocks) that have divergent erasure counts [31]. Additionally, bad block management is performed on-device. Retired blocks are hidden via an indirect layer between the raw flash blocks and pseudo physical blocks [32, 33]. For a given pseudo physical block address, the mapping is an identity mapping if the corresponding physical block is intact, but when a physical block is retired, the corresponding pseudo physical block is remapped to a spare block from an overprovisioned pool.

Importantly, previous work has shown that a block abstraction can be implemented on top of the ZNS interface [4, 34, 35] or OCSSD interface [6]. Also, ZNS can be built on top of OCSSD [7]. This suggests that an FTL implementing the functionality necessary to expose a block interface can be viewed as implicitly containing the internal machinery that ZNS and OCSSD expose explicitly. From this observation, we arrive at a general question, which we address in Section 4: can we characterize the FTL design space under a framework that cleanly separates FTL mechanisms and flash management policies, so that different storage semantics (e.g. OCSSD, ZNS, block interface) and the policies can be exposed dynamically?

3 Models and Assumptions

We consider a computing device equipped with a NAND-based NVMe SSD (Figure 1). We assume that the SSD correctly runs the firmware that implements our SxSSD design. This assumption is reasonable because 1) the independent flash storage hardware (processor, memory) provides isolation between the FTL firmware and the host OS, and 2) methods such as secure boot [36, 37] can prevent unauthorized firmware modification. We do not consider data loss in the DRAM cache due to power failure as modern SSDs can be equipped with capacitor or battery backed DRAM [27, 28].

We consider that the computing device runs trusted applications controlled by a trusted system administrator. The integrity and isolation of these applications can be maintained by running them in a secure environment such as a trusted execution environment [24, 38, 39], secure bootloader [40], or remote trusted computer [41]. These mechanisms can prevent applications from being tampered with even in the presence of a compromised operating system. We assume that all entities authorized to manage policies are mutually trusted.

We assume that the host OS can be compromised by an adversary capable of performing privilege escalation and obtaining root access. Hence, the host OS is untrusted. We model this as a Dolev-Yao-style adversary which controls the communication channel between the trusted application and the interface exposed by the SSD, yet cannot feasibly break cryptographic primitives [42]. This implies that the host OS has full access to the interface exposed by the SSD, at any time. In particular, the adversary may be active while the system administrator is accessing the policy management interface (i.e. meta-interface).

We also assume that the SSD contains a device-specific key-pair, generated and embedded by the manufacturer, as assumed in previous work [39, 22]. A certificate associated with this key is provided by the manufacturer to the trusted system administrator. In addition, the system administrator generates a key-pair before obtaining the SSD. Through a secure communication channel with the manufacturer, the administrator's public key is certified. The resulting certificate is provisioned to the SSD. Importantly, this grants the system administrator the ability to install custom policies onto SxSSD, as detailed in § 5.2. The system administrator directly controls a trusted application that can (1) install policies onto SxSSD, and (2) interact with those policies through their exposed interfaces.

4 A Characterization of the FTL Design Space

In this section, we answer the question posed at the end of Section 2.3. We identify common structural properties across the major FTL designs and examine how different FTL designs expose these properties to the OS (Section 4.1). Based on the observations made in Section 4.1, we define a new conceptual model for our SxSSD design (Section 4.2).

This new model allows trusted applications to dynamically define FTL policies so that the flash-to-OS interface turns changeable.

4.1 Common Structural Properties of FTLs

4.1.1 Internal Parallelism and Sequential Write Domains

Our first observation is that FTLs internally stripe writes across some hierarchy level (i.e. channel, die, plane, block, or page) to exploit device-level parallelism. Additionally, we observe that block-interface, open-channel, and ZNS SSDs manage the internal striping through sequential-write constraints, but differ in (1) the size of the "striping domain", (2) the striping level, and (3) how they expose this to the host. Illustrating (1) and (2), as discussed in Section 2.3, OCSSDs typically write data in chunks, striping successive writes across planes within a die, whereas ZNS and block-interface SSDs typically write into superblocks that stripe successive writes across channels first (see Figure 2). Then, demonstrating (3), block-interface SSDs hide the sequential-write constraint behind an additional layer of indirection through a page-level mapping table.

To make this precise, we define a logical address space as any address space constructed inside the FTL above raw physical flash, independent of whether it is exposed to the host. A striping domain is a group of addressable units in an address space across which the FTL advances writes to exploit internal parallelism, with writes ultimately striped over an underlying group of physical pages (e.g. a superblock). A sequential write domain (**SWD**) is the logical view of a striping domain: writes are sequential with respect to the SWD's logical write pointer, even though the underlying striping domain may place consecutive logical writes across multiple channels, dies, and/or planes. For example, a ZNS zone can be viewed as an SWD with a superblock as its striping domain.

Given a striping level and SWD size, an address space can be partitioned into SWDs. At the lowest level, the raw flash consists of physical blocks. On top of this, bad block management (BBM) introduces a pseudo-physical address space by mapping pseudo-physical blocks to physical blocks. Each pseudo-physical block is then a striping domain with page-level striping. Since an SWD is the logical view of a striping domain, each pseudo-physical block has a corresponding "primitive" SWD (we call these *pSWDs*). Higher-level SWDs exposed to FTL policy (we call these *eSWDs*) may then be composed of multiple pSWDs. Here, *exposed to policy* refers to being visible to the internal FTL policy logic, independent of how the policy exposes this through the host interface.

From the above, we see that OCSSDs expose SWDs composed of $planes\_per\_die$ pSWDs with plane-level striping, whereas ZNS and block-interface FTLs use SWDs that span $planes\_per\_die \times dies\_per\_channel \times num\_channels$ pSWDs with channel-level striping (i.e. superblocks). The key difference between ZNS and block-interface FTLs is how these eSWDs are exposed *to the host*: ZNS exposes SWDs directly to the host as zones, while block-interface FTLs instead present a flat logical address space and map host writes onto these eSWDs through a page-level mapping table.

Across all designs, SWDs are associated with state machines that maintain internal state, such as mapping, wear status, write pointers, and SWD state (e.g., free, open, closed). For SWDs exposed to policy, this state can be extended with policy-defined state to realize particular semantics. A subset of the resulting state, selected by policy and relevant to the interface, may become host-visible, while the remainder stays device-internal.

4.1.2 Event-Driven Policies

Our second observation is that all policy-level actions are naturally event driven, because the normal FTL functions (e.g. mapping, garbage collection, wear leveling, and bad block management (BBM)) take place in response to changes in the device state. That is, policies are triggered when certain conditions over state maintained by the per-SWD state machines are satisfied. Additionally, this state changes during specific "events" such as writing a new page, erasing a block, etc. As an example, the per physical block wear status is tracked by the associated pSWD state machine. Wear leveling (e.g., included in ZNS and block interface SSDs) is then triggered when erase counts diverge beyond some threshold, with this condition evaluated on write, erase, or background events. On the other hand, OCSSDs simply report wear status to the OS which performs wear leveling manually. Additionally, as mentioned above (§ 2.3.3), BBM policies respond to physical block failure events by either remapping the affected pseudo-physical block to a healthy block, or retiring it. We can model retirement (used by the OCSSD and ZNS BBM policy) as a degenerate case of remapping, in which the pseudo physical block is remapped to null. Together, these examples illustrate that *diverse FTL behaviors can be modeled as event-driven actions triggered by conditions over the SWD state*. This naturally extends to other FTL functions such as garbage collection and address translation.

4.2 A Conceptual Model of our New FTL Design

The structural properties identified in Section 4.1 suggest that FTL designs share a common underlying organization where writes advance over SWDs with associated state machines, and policy-level behavior is triggered by events over this state. Based on this key observation, we define a new conceptual model for the FTL design. Specifically, we can model an FTL as being composed of five conceptual components: **(M1)** the raw flash storage, consisting of the physical geometry constrained by C1-C4; **(M2a)** a mechanism layer that constructs a pseudo-physical address space through bad block management (BBM) and defines the primitive SWDs (*pSWDs*) over that space; **(M2b)** a mechanism layer that groups pSWDs into higher-level SWDs exposed to policy (*eSWDs*); **(M3)** a set of events, triggered either internally or by specific NVMe commands; **(M4)** the policy-defined eSWD geometry, context, and state machines, and **(M5)** a set of policies defined as pairs of (condition, action) functions attached to events.

To make the FTL extensible, we fix M1-M3 as mechanisms, and add the meta-interface, which makes M4-M5 (policy) programmable, enabling customization of the host-visible interface semantics. We expand on the definitions of M1-M5 below:

Similarly to **(M1)**, any FTL is built on top of raw flash storage hardware that exhibits constraints C1-C4. Thus, M1 consists of the read/write/erase operations consistent with these constraints.

Corresponding to **(M2a)**, the BBM indirection layer introduces a *pseudo-physical* address space, where each pseudo-physical block is a block-sized striping domain with page-level striping. Then, each pseudo-physical block creates a pSWD (thus, the striping domain of a pSWD is a pseudo-physical block). BBM then maintains a mapping from pseudo-physical to physical blocks, with overprovisioned physical blocks initially left unmapped. The BBM layer exposes an API to policies for maintaining the pseudo-physical to physical block mapping over time, and migrating valid data when a pseudo-physical block must be remapped to a new physical block. As established in Section 4.1.2, this model is sufficient to implement the policies of ZNS, OCSSD, and block-interface FTLs.

pSWD state and context.

- **Mapping:** The physical block currently mapped to the underlying pseudo-physical block.
- **Wear status:** Erase count of the mapped physical block.
- **pSWD state:** Free, open, closed.
- **Write pointer:** The next page to be written.
- **Page validity:** Per-page status (free, valid, or invalid).

Corresponding to **(M2b)**, the next mechanism layer is a higher-level abstraction over the pSWD space to policies. This abstraction, called an exposed SWD (eSWD), groups pSWDs into policy-visible units over which writes are striped according to the striping domain and striping level selected by the policy.

The API exposed to policy for managing eSWDs provides four key capabilities: 1) accessing and changing the mapping between pSWDs and eSWDs, 2) migrating data between eSWDs, 3) issuing read, sequential write, and erase operations over eSWDs, and 4) managing the eSWD context.

Mechanism-level eSWD context.

- **Mapping:** A mapping to the pSWDs covered.
- **Layout:** The policy-defined eSWD size and striping level.
- **Write pointer:** The next page to be written.

The FTL events **(M3)** trigger specific policy functions when they occur and so are the mechanisms through which policies are called.

FTL event types (M3).

- The background processing event.
- NVMe commands.
- pSWD state transitions.
- Error events over primitive flash operations.

For **(M4)**, policy defines the logical organization and semantics of eSWDs. As mentioned above, this includes defining the size and striping level of eSWDs, which determines how M2b maps logical writes onto the underlying pseudo-

physical space. Policy also defines any additional eSWD-level context and state machines needed to realize a particular flash-to-OS interface. In this way, M4 captures the storage semantics that distinguish one FTL interface from another.

For example, a block-interface policy uses eSWDs spanning superblocks with channel-level striping. Additionally, this policy maintains a page-level mapping table. On NVMe write events, the block-interface policy would write the new page at the write pointer of the current open eSWD, update the mapping table to point to the new physical location, and invalidate the old physical page.

Then, corresponding to **(M5)**, policies are expressed as (condition, action) pairs stored in function pointer tables attached to events. Conditions are boolean functions defined in terms of policy specific data structures, state-machine status, and event-specific information. When a condition is satisfied, the corresponding action function is executed. The action interacts with flash storage through the policy-visible API (M2).

5 SxSSD Design

5.1 Design Overview

To enable programmability of both the host-visible storage interface exposed by SxSSD and the on-device policies that implement it, we propose a new FTL structure that realizes the conceptual FTL model defined in Section 4.2. Our resulting FTL structure consists of three main components, shown in Figure 3: (1) the **flash subsystem**, which instantiates M1-M2 and presents an API for policy code to call into; (2) the **policy engine**, which manages runtime policy state and context (M4), and executes policy code by routing events (M3) to the policy functions (M5) via dispatch tables; and (3) the **meta-interface**, which enables secure management of the policies that implement M4 and M5.

The meta-interface provides mechanisms for the upper layer to manage custom policies on the FTL. This includes installing, updating, temporarily deactivating, and removing policies. However, exposing this interface to the host OS without carefully restricting how upper-layer software may invoke it creates a critical attack surface. Accordingly, a significant responsibility of the meta-interface includes restricting access to policy management and ensuring that only the trusted application(s) can manage policies, even in the presence of an actively adversarial host OS with Dolev-Yao capabilities. Details of the meta-interface are elaborated in Section 5.2.

After a policy is stored, it must be loaded and enforced at runtime. The policy engine contains the policy storage and the mechanisms for loading policies, maintaining their state, and calling into them when events are triggered. Its core component is a set of dispatch tables, each associated with an event, that point to the corresponding policy handlers (i.e. (condition, action) pairs). Corresponding to M3, the events include receiving NVMe commands, state transitions in the primitive SWD state machines, errors on primitive flash operations, and the background event. The details of the policy engine are elaborated in Section 5.3.

To enable interaction between policy functions and flash storage, the flash subsystem implements the storage management mechanisms and exposes them to policy through a well-defined interface. We construct this subsystem by starting from primitive flash operations constrained by (C1-C4), and adding a few powerful abstractions on top: The raw flash operations are the lowest layer, performing read/write/erase on individual pages and blocks. The BBM indirection layer is implemented on top of the raw flash operations, while the eSWD abstraction and policy API are built on top of the BBM primitives. It is expected that regular policies only call into the policy API (Section 6.2). The details of the flash subsystem are elaborated in Section 5.4.

5.2 The Meta-Interface

The meta-interface provides the host with a small set of primitives for securely managing on-device policies. These functionalities are presented to the host as new vendor-specific NVMe commands, described in Table 1. Since each NVMe command triggers an event that indexes into the policy engine's dispatch table, the meta-interface is itself implemented as a privileged policy. To prevent an untrusted host from abusing this interface, the meta-interface policy should authenticate requests and ensure their integrity and freshness.

5.2.1 Authorized Policy Management with Replay Protection

Towards achieving this, we add an NVMe command `INIT_SESSION` that establishes a session between a trusted application and the SSD. Part of this process is to generate a shared session key K_s. Because both parties possess each other's certificates, this can be done securely using an authenticated key exchange protocol [43].

To secure the policy management NVMe commands, the meta-interface only accepts requests generated by entities holding K_s (i.e. a trusted application). This is enforced by authenticating each request using a message authentication

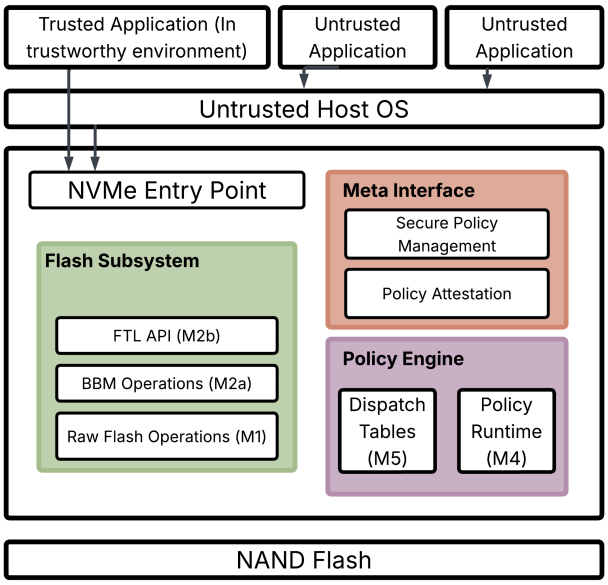

Figure 3: An overview of SxSSD. Note that events (M3) are generated across multiple components.

NVMe Command	Description
`INIT_SESSION`	*Resets the counter (ctr_0), establishes a session key (K_s) shared between a trusted application and SxSSD, and sets the session mode (normal or confidential).*
`INSTALL_POLICY`	*Installs a new policy into the overprovisioned policy storage area, and tracks relevant metadata.*
`UPDATE_POLICY`	*Replaces the code of an existing policy while preserving its policy ID.*
`REMOVE_POLICY`	*Deletes a policy and its associated metadata from the overprovisioned storage area.*
`ACTIVATE_POLICY`	*Activates a policy by reading it, parsing its code, then loading it into the policy engine.*
`DEACTIVATE_POLICY`	*Deactivates a policy by removing its functions from the policy engine.*
`POLICY_ATTESTATION`	*Generates an authenticated attestation report on a policy's installation status, activation state, and optionally its code integrity via hash verification.*

Table 1: Vendor-specific NVMe commands introduced by the SxSSD meta-interface. All meta-interface commands (except `INIT_SESSION`) require authentication with K_s and include counter-based replay protection. In confidential mode, request data are encrypted.

code under key K_s. Importantly, this enables the FTL to check both the source and integrity of any request data that are covered by the MAC. The attacker might manipulate the NVMe opcode itself, so that both the opcode and request data should be authenticated. As an initial solution, before calling the NVMe commands (Table 1, excluding `INIT_SESSION`), the trusted application must compute a MAC over the NVMe opcode concatenated with the request data. The FTL can then verify the MAC correctness before continuing to handle the relevant NVMe command.

Unfortunately, this method does not prevent an adversary from replaying valid policy management requests. To prevent this, we introduce a counter ctr_0 maintained by the meta-interface that is reset whenever a call to `INIT_SESSION` succeeds. Correspondingly, the trusted application maintains a counter ctr_1. Then, on any call into the meta-interface, the trusted application will include ctr_1 in the request data that is covered by the MAC. In turn, the FTL enforces that $\mathsf{ctr}_1 > \mathsf{ctr}_0$ before authenticating the request, and sets $\mathsf{ctr}_0 \leftarrow \mathsf{ctr}_1$ afterwards. This ensures that even if the original contents of a request are identical, either the session key or the counter value included in the MAC computation

must be unique for a request to be authenticated. In particular, this also ensures *policy freshness*, since invocations of `INSTALL_POLICY` and `UPDATE_POLICY` cannot be replayed to downgrade active policies.

5.2.2 Policy Confidentiality

During policy management, important policy-related request data are passed through the host OS to the meta-interface (e.g. the policy code or parameters). However, an adversarial OS may use this information to infer the policy semantics, which will compromise security guarantees for some applications. This is especially relevant for specific policies, such as those designed for plausibly deniable storage systems [21].

To prevent the adversary from learning policy semantics when calling into the meta-interface, we introduce a new session mode called the confidentiality mode (in contrast to normal mode). The trusted application enters this mode as part of calling `INIT_SESSION`. During this mode, confidentiality is ensured over policy request data for each meta-interface interaction by encrypting request data. To achieve this, the policy confidentiality mode replaces the MAC-based authentication mechanism with an authenticated encryption with associated data (AEAD) scheme, which ensures confidentiality, integrity, and authenticity of request data [44]. As the parameters to the authenticated encryption algorithm, the trusted application uses K_s as the key, $\mathtt{ctr}_1$ as the nonce, the request data as the plaintext, and the NVMe opcode as the associated data. Similar to the normal session mode, the FTL enforces that $\mathtt{ctr}_1 > \mathtt{ctr}_0$ before performing authenticated decryption of the request.

5.2.3 Policy Protection at Rest

Calling the meta-interface commands `INSTALL_POLICY` and `UPDATE_POLICY` stores the plaintext policy code on the SSD, so that they can be easily loaded by the policy engine. To ensure both policy confidentiality and integrity at rest, we store policy code in a reserved on-device region that is not reachable through the host-visible logical address space. This is challenging since policies will present different logical addressing schemes to the host OS, and a policy may map the region containing policy code into the host-visible address space.

To address this, the key idea is to store policy code in a region that is not visible to the policies themselves (i.e., cannot be mapped to eSWDs). To achieve this, policy code is stored within a portion of the SSD's over-provisioned space. Since this space is not mapped to any pseudo-physical block address, it will not be exposed to the host by policies, which only interact with the flash storage through the FTL API. The remaining problem is to prevent policies from eventually mapping this overprovisioned space onto the policy-visible eSWDs. To do this, the BBM layer simply refuses to map blocks containing policy code into the policy-managed pseudo-physical address space, preventing this area from being exposed. To manage this storage space, the meta-interface policy uses privileged functions not available to normal policies, interacting directly with the flash storage through the raw flash operations.

5.2.4 Policy Attestation

At any point, the system administrator expects a specific set of policies to be installed and active based on the sequence of meta-interface interactions. However an adversarial host OS may manipulate the communications between any trusted application and the meta-interface. Since any synthetic or modified commands will be rejected, the adversary is limited to causing valid requests to become invalid and be rejected, or preventing them from reaching the SSD. While this cannot be prevented under the Dolev-Yao model [42], we enable the system administrator to detect mismatches between the expected and actual on-device policy state.

To support this, we introduce the meta-interface command `POLICY_ATTESTATION`, which is a request for the FTL to generate a report on the status of a given policy. The request data for `POLICY_ATTESTATION` simply consists of a policy ID, which is maintained by the meta-interface along with other bookkeeping metadata such as the location and size of the policy as part of policy installation. We introduce two report types, depending on the policy properties being audited.

The first report type is called the security report, and it consists of generating a report over two bits of information, which is sufficient for security. Specifically, the report is generated over (1) a bit indicating whether the policy code associated with the provided ID is installed, and (2) a bit indicating whether the policy is active or not. This is sufficient under our threat model because (i) only authenticated meta-interface commands can install or modify policies, (ii) policy storage is integrity-protected and inaccessible to the host-visible address space. From these, we conclude that policy installation implies policy integrity. Security reports are then generated by computing a MAC or AEAD over the two bits, depending on whether the current session is in normal or confidential mode, respectively.

The second report type is called the consistency report. While the security report is sufficient under our threat model, system-level faults such as firmware bugs or flash corruption may cause policy code to be partially written, lost, or

corrupted. Therefore, the consistency report replaces the first bit in the security report with a hash of the relevant policy code. To verify such a report, the trusted application should store a hash of the expected policy code.

5.3 Policy Engine

The policy engine is responsible for ensuring that installed and active policies are enforced by the FTL. When the FTL boots, the SxSSD firmware is loaded into DRAM and begins execution. However, any "active" policies are stored in flash-memory but not yet loaded into system RAM. To do this, the policy engine is invoked on startup, and it references the policy metadata, identifying policies with the active bit set and loading their code into RAM. As established above, (Section 4.2) policies include special functions that are called when certain "events" occur in the FTL. Corresponding to M3, policies can register functions to four event types: The background event, NVMe commands, pSWD state transitions, and primitive flash operation errors. The background event is triggered by the policy engine, NVMe command events by the host OS, and pSWD state transitions and flash operation errors by the flash subsystem. Policies are essentially sets of (condition, action) pairs, implemented in a general purpose programming language, and pointers to these need to be registered with the correct event dispatch table. As a convention, each policy will include a function called `INIT_POLICY`, which the policy engine calls immediately after loading the policy code into RAM to perform this registration.

During runtime, each event calls a `DISPATCH` function, which refers to the table associated with that event. Then, for each active entry in the table, `DISPATCH` will first execute the condition function, then if the condition is satisfied, the action function will be executed. The condition and action functions can call into the FTL API, which is part of the flash memory subsystem, to interact with the storage system.

5.4 Flash Subsystem

The flash subsystem is the part of SxSSD through which policies interact with the flash storage. To distinguish between the device-enforced constraints and policy enforced constraints built on top of them, we split the flash subsystem into three layers, corresponding to M1, M2a, and M2b, respectively: (1) the raw flash operations, (2) the bad block management, and (3) the FTL API.

5.4.1 Raw Flash Operations.

The lowest layer interacts directly with the flash storage hardware and ensures that all operations satisfy constraints C1-C4. In this layer, the operations are performed directly over physical blocks and pages. Any errors are reported as events to the policy engine. Then, policies handling such events may invoke BBM functions exposed in the FTL API.

5.4.2 Bad Block Management and pSWDs

On top of the raw flash operations is the BBM subsystem, which realizes M2a. BBM introduces a pseudo-physical block address space and maintains the mapping from pseudo-physical block addresses to physical blocks. Since each pseudo-physical block is a block-sized striping domain with page-level striping, it is associated with one primitive SWD (pSWD). Thus, this layer maintains the pSWD state machines and context. In addition, BBM provides the functions needed to manage the pseudo-physical to physical mapping, and includes wrappers around the raw flash operations (i.e. pSWD read, write, and erase operations that handle the mapping transparently). Over-provisioned blocks are managed at this layer as physical blocks with no pseudo-physical mapping, which makes them inaccessible through the address space exposed to policies.

BBM also provides a configuration function, `SET_OP_SIZE`, which determines how much physical space is reserved for overprovisioning. This function is exposed to policies through the FTL API and is intended to be invoked by a single designated management policy during `INIT_POLICY`. Over-provisioned blocks are distributed evenly across planes to preserve flash-level parallelism after remapping. In addition, the reserved space must also be sufficient to store the policies installed and their metadata.

Finally, BBM provides the basic mechanisms needed for bad-block handling and remapping. When a flash operation error event indicates that a block should be retired, policy can choose to call BBM functions that retire the affected physical block, update the pseudo-physical mapping, and migrate any associated data.

5.4.3 FTL API and eSWDs

The top layer of the flash subsystem realizes M2b. It consists of all the functions available to be called directly by policies, as well as the supporting mechanisms required to implement them on top of BBM. Importantly, this layer

introduces the eSWD abstraction, which groups pSWDs into policy-visible eSWDs. The FTL API exposes generic read, sequential write, and erase operations over eSWDs, translating each such request into the corresponding BBM operations over the underlying pSWDs. The device-side eSWD context includes the eSWD size, striping level, per-eSWD write pointer, and the mapping from eSWD onto pSWDs. In particular, the write pointer determines the placement of new writes within the eSWD and enforces the sequential-write requirement.

The FTL API enables policy to determine the eSWD size and striping level via two important functions: 1) `SET_ESWD_SIZE` sets the eSWD size, thereby determining how many eSWDs exist. 2) `SET_STRIPING_LEVEL` sets the striping level, which determines the layout of each eSWD in the pseudo-physical address space, and how its write pointer advances over the associated pSWDs. These functions should be called during `INIT_POLICY` by exactly one policy.

The FTL API provides functions for querying and managing the eSWD to pSWD mapping. Also, it supports functions such as wear-leveling, higher-level mappings that enable random writes over a logical address space (such as a standard logical block interface), and garbage collection by providing primitives for migrating eSWDs and abstractions for creating and managing additional mapping tables.

6 Analysis and Discussion

6.1 Security Analysis

Security Requirements of Extensible SSDs. Supporting extensibility under a strengthened threat model introduces additional security requirements that do not arise in fixed-function SSDs. In an extensible SSD, a new interface must be introduced through which external entities can influence FTL execution. In SxSSD, this interface is realized as the meta-interface, described in Section 5.2. However, under our threat model (Section 3), the meta-interface is always mediated by an untrusted host operating system. The essence of the security challenge is therefore to precisely constrain the conditions under which accessing this interface is allowed to affect FTL execution. Under our threat model, the meta-interface should be accessible only to the trusted system administrator or to applications explicitly delegated by the administrator. Our six security aims are a direct consequence of this observation: (1) Entities attempting to access meta-interface commands must be authenticated, and (2) all legitimate meta-interface interactions must be fresh and non-replayable. This effectively prevents untrusted entities from directly invoking any call into the meta-interface. (3) Meta-interface invocations must be integrity protected such that any modification by the host OS while forwarding requests to the SSD is detectable. (4) Meta-interface invocations must be confidential to prevent the host OS from learning policy semantics. (5) To preserve the confidentiality and integrity of installed policies over time, all interactions with the policy state must be mediated exclusively through the meta-interface. (6) While we cannot prevent DoS attacks under a Dolev-Yao-Style adversary, we must ensure that the meta-interface remains effectively available to the system administrator by making any inconsistencies between the expected and actual system states detectable.

SxSSD **Security Mechanisms.** Our design achieves requirement (1) because the meta-interface verifies either a MAC or AEAD generated using K_s upon every meta-interface invocation. Clearly, an adversary that does not hold K_s cannot generate a valid MAC or AEAD except with negligible probability. Our design achieves requirement (2) because the MAC/AEAD authenticates the session counter ctr_1 and enforces its uniqueness within a session. As a result, any valid MAC/AEAD is either generated under a unique session key or computed over unique data. Requirement (3) follows as a direct consequence of verifying a MAC/AEAD over the request data. SxSSD achieves requirement (4) by introducing normal and confidential session modes. In normal mode, requests are authenticated using a MAC, whereas in confidential mode, requests are protected using AEAD. We achieve requirement (5) by storing policy to the over-provisioned area, and excluding it from any pseudo-physical block mapping. Since the eSWDs map onto the pseudo-physical address space, no storage interface specified by policy can modify or read the policy storage. In principle, this invariant could be violated if policy code invokes functions outside the FTL API. However, since policy code is provided by a trusted system administrator under our threat model, such violations constitute safety failures rather than security violations. We discuss potential mitigations in § 6.2. Requirement (6) is met by our `POLICY_ATTESTATION` meta-interface command, which allows the system administrator to query policy state at any time. Under our threat model, the adversary neither controls an authorized policy manager nor possesses K_s, and therefore cannot authenticate policy changes between attestations. Because requirements (1)–(3) and (5) are satisfied, the presence of an installed policy implies that its integrity has been maintained. Therefore, the only state that must be attested is (i) whether a policy is installed, and (ii) whether it is active. Directly attesting policy integrity falls under safety rather than security, since under (1)–(3) and (5), only faulty trusted policy code can corrupt policy storage. Nevertheless, we also support an attestation mode in which policy integrity is explicitly reported.

Parameter	Baseline	SxSSD
Channels	8	8
LUNs per channel	8	8
Planes per LUN	1	1
Blocks per plane	256	276
Pages per block	256	256
Page size	4 KiB	4 KiB
Physical capacity	16 GiB	17.25 GiB
Policy-visible capacity	16 GiB	16 GiB
Overprovisioning	0%	7%
Page read latency	40 μs	40 μs
Page program latency	200 μs	200 μs
Block erase latency	2 ms	2 ms

Table 2: Device configuration used in our experiments. Notably, SxSSD includes internal overprovisioning while keeping the policy-visible capacity unchanged.

6.2 Discussion

Policy Code Safety. As discussed in Section 6.1, we assume that the system administrator is trusted, allowing us to treat policy code installed on the FTL as benign. However, policy code may still violate the development guidelines by invoking functions outside the FTL API, resulting in potential safety failures. Such violations may corrupt policy storage or reveal policy semantics. In addition, policy code may also unintentionally contain vulnerabilities, such as buffer overflows [45] or use-after-free errors [46]. A promising future direction is to explore the mitigation of these errors through static analysis [47], or restricting policy capabilities during runtime by integrating eBPF into the storage device [48].

Policy management by mutually distrusting entities. Our threat model assumes that all entities authorized to manage policies are mutually trusting. However, there may be scenarios in which a trusted application may need to install a policy while distrusting another authorized policy manager. In this setting, the current `POLICY_ATTESTATION` functionality may be insufficient, as only the *current* policy state is audited. A distrusted manager could temporarily replace and later restore the active policy between attestations, violating the security guarantees expected by the trusted application. Supporting this model would require new attestation mechanisms that provide authenticated evidence of policy transitions over time. We leave this to future work.

Adapting SxSSD to other SSD interfaces. Although the SxSSD meta-interface uses vendor-specific NVMe commands, it can be ported to SATA/SAS using their vendor-specific command mechanisms, without changing the policy engine or flash subsystem significantly. Additionally, policies would have limited capability to create custom vendor-specific commands, as SATA can support 27 [49] and SAS can support 64 [50] in total.

7 Implementation and Evaluation

Our evaluation is centered around three questions: **(Q1)** How efficient are the secure policy management functions? **(Q2)** What is the performance overhead of SxSSD compared to native FTL implementations? **(Q3)** Can SxSSD support different security-critical storage semantics with minimal implementation effort and overhead comparable to native implementations?

We address **(Q1)** in Section 7.3, by measuring the execution time of the vendor-specific NVMe commands introduced by the meta-interface (Table 1). We address **(Q2)** in Section 7.4 by implementing a block-interface policy on top of SxSSD and comparing it against FEMU's native block-interface implementation. To establish a fair comparison, we first demonstrate that the two implementations provide matching semantics by using the traces listed in Table 3 to compare their internal behavior. We then compare throughput and latency to measure the baseline overhead introduced by SxSSD. In Section 7.5, we address **(Q3)** by implementing FlashGuard [19], an FTL-based ransomware recovery strategy, as a modification of our block-interface policy. We also quantify implementation effort by reporting the lines of code required for this modification. We compare the additional overhead introduced by this modification with the overhead reported in the original FlashGuard paper.

Family	Workload	ID	Write Ratio
Systor [52]	Systor LUN 5	S5	49.13%
	Systor LUN 6	S6	39.16%
	Systor LUN 11	S11	40.19%
MSR [53]	Proxy server	M-prxy	98.36%
	Print server	M-prn	95.34%
	Media server	M-mds	92.18%
FIU [54]	Web server	F-web2	99.98%
	Mail server	F-mail3	99.11%
	Homes server	F-homes2	99.29%
Alibaba [55]	Alibaba device 1	A1	100.00%
	Alibaba device 8	A8	100.00%

Table 3: Summary of evaluated workloads.

7.1 Implementation

We have implemented SxSSD on FEMU [25], which is a QEMU-based flash emulator used in many research projects [13, 22]. Our implementation and evaluation artifacts are publicly available [51] at commit `3451afc74`. For the baseline comparison, we implemented a block-interface policy that matches the semantics of the original FEMU implementation (we call this SxSSD-Block) in 543 lines of C code. We also implemented a FlashGuard [19] policy (we call this SxSSD-FlashGuard) as a case study, which is a flash-based ransomware recovery scheme. This policy is built on SxSSD-Block and adds 533 additional lines of code. Table 2 shows the configuration of the simulated device. The only configuration difference is that SxSSD has 7% extra physical capacity to serve as overprovisioned blocks. A small portion of this space is used to store the policy code (i.e., just 6 pages for SxSSD-Block). Otherwise, the policy-visible and host-visible capacity and geometry are identical to the baseline (i.e., the overprovisioning provides no practical advantage other than providing space for policy code storage). As our evaluation machine, we use a desktop computer with an Intel Core Ultra 7 265K 3.9 GHz and 64GB DDR5 4800 MT/s RAM. To simulate a low-power flash memory controller, we scaled the clock rate to 0.5 GHz from 3.9 GHz by multiplying the *computation* time by 7.8, similar to the methodology used in [22]. Finally, the cryptographic primitives were instantiated as follows: the authenticated key exchange for session establishment uses X25519-based Diffie-Hellman and Ed25519 signatures. The key derivation function is HKDF-SHA256, the MAC uses HMAC-SHA256, the hash function is SHA-256, and the authenticated encryption with associated data (AEAD) function uses AES-GCM.

7.2 Experimental Setup

In our experimental setup, the host operating system is Arch Linux with Linux kernel 6.19.11. To build and launch FEMU, we use an Ubuntu 24.04 container managed by Distrobox with the Docker backend. FEMU runs an Ubuntu 20.04 guest virtual machine with the Linux kernel 5.15.0, configured with 4 GB of RAM and 4 vCPU cores. To avoid performance unpredictability, we set the host CPU to "performance" mode, and pin the vCPUs to physical CPUs, using the FEMU-provided scripts [56].

7.2.1 Workloads and Evaluation Method

In our experiments, we use the FIO benchmarking tool [57] to run real-world workload traces and throughput benchmarks. We use workload traces from the SYSTOR'17 dataset [52], Microsoft [53], FIU [54], and Alibaba [55]. We selected the subset of the trace datasets shown in Table 3 with the highest write ratios. Additionally, we scale the workload request addresses to fit in the host-exposed logical address space, and cap the time between requests at 10ms to save evaluation time. When running the workloads, we split into a warmup phase and an experiment phase: in the warmup phase, we first run FIO random write across 70% of the logical address space, then run the first 1,000,000 operations of the workload, and for the experiment phase, we capture the timing information during 100,000 additional operations. Before running each workload, we relaunch the VM, then install and activate the relevant policy through the meta-interface (when evaluating SxSSD).

7.3 Meta-interface Performance

The execution time of the vendor-specific NVMe commands introduced by the meta-interface is shown in Table 4. The evaluation is split into normal mode and confidential mode; overall, the time difference between these modes is negligible. Notably, the consistency-mode attestation has 41% higher execution time than security mode in both normal and confidential modes, though both operations are still trivially fast. This is expected, since the consistency mode

Command	Normal (ms)	Confidential (ms)
INIT_SESSION	1.337	1.132
INSTALL_POLICY	0.927	0.920
UPDATE_POLICY	1.492	1.468
REMOVE_POLICY	0.881	0.896
ACTIVATE_POLICY	128.777	132.965
DEACTIVATE_POLICY	1.177	1.154
POLICY_ATTESTATION (S)	0.223	0.244
POLICY_ATTESTATION (C)	0.315	0.344

Table 4: Average device-side execution time of meta-interface commands in normal and confidential modes. The POLICY_ATTESTATION (S) is in security mode, while the POLICY_ATTESTATION (C) is in consistency mode.

Internal Behavior	S11	S6	S5	M-prxy	M-prn	M-mds	F-web2	F-mail3	F-homes2	A8	A1
Physical page writes	0.000	-0.027	0.000	0.006	0.042	0.006	0.000	0.002	0.000	0.001	0.028
GC invocations	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
GC pages migrated	0.000	-0.342	0.000	4.189	1.174	11.487	0.000	11.444	0.000	0.035	5.219
Block erases	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000

Table 5: Semantic equivalence. Each entry reports the percent difference (%) between Baseline and SxSSD-Block.

computes a hash over the entire policy code, whereas the security mode only needs to validate two bits of information. By far the most expensive meta-interface operation is ACTIVATE_POLICY, because it has to 1) read the policy code into memory, 2) parse the policy binary resolve its symbols, and 3) execute the INIT_POLICY function, which configures the device geometry and loads the policy into the policy engine.

7.4 Baseline Performance Overhead

7.4.1 Baseline and SxSSD-Block Policy Semantic Equivalence.

To ensure that our performance comparison is fair, we first verify that the SxSSD-Block reproduces the same internal behavior as the baseline implementation. Table 5 reports the percent difference between the baseline block-interface FTL and SxSSD-Block for several internal behaviors across all workload traces. First, baseline and SxSSD perform the same number of GC invocations and block erases for every workload, indicating that 1) garbage collection is triggered under the same conditions and 2) the units of garbage collection (i.e., a superblock-sized eSWD) are identical.

Additionally, the physical page write counts differ by at most 0.042%, indicating negligible differences in write-amplification. Although some workloads show larger percent differences in GC pages migrated in Table 5, Table 6 shows that these occur only when little garbage collection takes place: the average migration-count difference is 0.52% for workloads with more than 100,000 migrated pages, but 4.04% for workloads with fewer than 100,000. These differences can be accounted for by the random writes included in the warmup workload causing variations in the overwrite distribution. Overall, these results indicate that the block-interface policy faithfully reproduces the baseline behavior, enabling a fair performance comparison between them.

7.4.2 Baseline Overhead.

Next, we compare the performance of the baseline implementation, SxSSD-Block. Since the policy semantics are the same, this captures a reliable measure of the overhead introduced by SxSSD overall. Figure 4 shows the average latency to complete a single write operation for each workload trace. Across all workloads, the average latency introduced by SxSSD is only 3.74%. Corresponding to this, Figure 5 shows that throughput is also slightly impacted. While random-read IOPS decreases by only 0.069%, random-write IOPS decrease by 4.617% relative to the baseline. These results indicate that SxSSD imposes negligible overhead on reads and a small overhead on writes.

7.5 Case Study

7.5.1 Recovery From Ransomware Attacks.

FlashGuard was the first work to introduce data recovery from malware attacks leveraging the out-of-place update behavior of block-interface FTLs [19]. It tracks recently read logical pages, and if such a page is later overwritten,

GC Pages Migrated	S11	S6	S5	M-prxy	M-prn	M-mds	F-web2	F-mail3	F-homes2	A8	A1
Baseline	0	585258	4246	1753	100378	919	0	173	0	113314	19278
SxSSD	0	583258	4246	1828	101563	1031	0	194	0	113354	20311
Percent difference	0.000	-0.342	0.000	4.189	1.174	11.487	0.000	11.444	0.000	0.035	5.219

Table 6: GC pages migrated by Baseline and SxSSD.

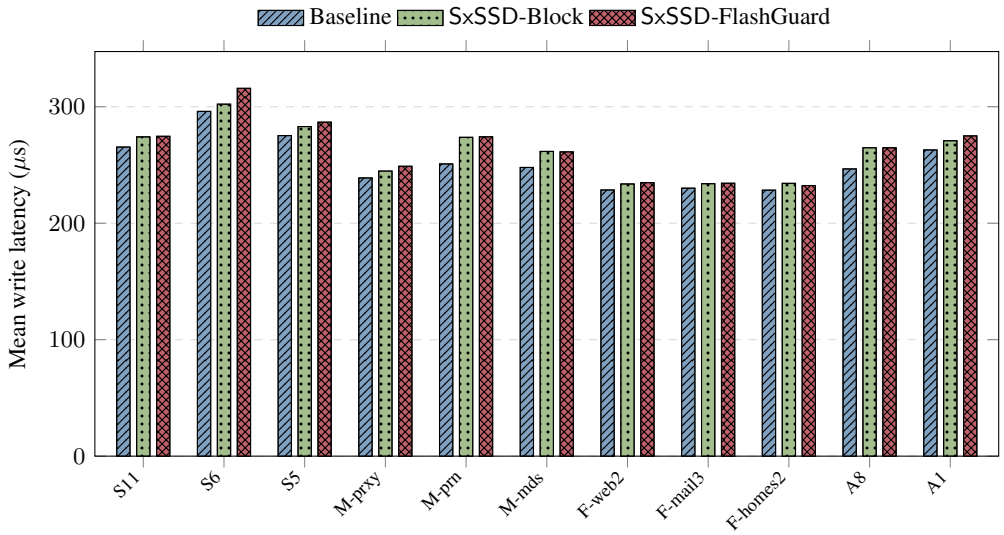

Figure 4: Mean write completion latency across evaluated workloads for baseline, SxSSD-Block, and SxSSD-FlashGuard.

FlashGuard treats the old version as suspicious and preserves it during garbage collection for recovery after an attack. In order to later restore retained pages after an attack, FlashGuard stores metadata such as the associated logical page address and previous (pseudo-)physical page address in the per-page OOB area.

We implemented SxSSD-FlashGuard as a modification of SxSSD-Block. Notably, we achieved this with only 533 additional lines of policy code, and without changing the FTL itself. The logic for retaining the previously read pages is implemented in the policy code attached to existing SxSSD events. In particular, the read behavior is implemented by modifying the function attached to the read NVMe command event. This change is to mark the corresponding bit in a read bitmap before passing control to the normal block-interface read function. The write behavior is implemented similarly by modifying the function attached to the write NVMe command event. When a logical page is written, it first checks whether the page has been read. If so, the policy marks the associated pseudo-physical page as retained. Otherwise, the page is invalidated normally. In addition, every write adds the necessary OOB metadata before allocating and writing to a new pseudo-physical page. The garbage collection, which is registered to the background event, is modified by migrating retained pages in addition to valid pages, and copying the OOB along with migrated pages. Additionally, the FlashGuard recovery interface is exposed through two policy-defined vendor specific NVMe commands. Overall, this demonstrates that FlashGuard is naturally expressible as a policy on SxSSD.

Next, we briefly evaluate the additional overhead introduced by the FlashGuard policy. We ran all workloads on SxSSD-FlashGuard, and the results are displayed in Figure 4. Relative to SxSSD-Block, the mean write completion latency increased by 4.50% at most, for the S6 workload. However, the average increase in latency across all workloads is only 0.86%. This difference arises because most of the workloads lack 1) sufficient read requests and 2) sufficient garbage collection. In the FlashGuard paper, the average latency increases by at most 6.1% [19], which is consistent with our results.

8 Related Work

Software-Defined and Extensible SSDs. Ouyang et al. [3] and Zhang et al. [58] were among the first to introduce software-defined SSDs. Their strategy was for the SSD to expose an interface close to the flash hardware characteristics and move FTL functions into the host OS. This work is a conceptual precursor to LightNVM [6], which introduced a Linux subsystem for open-channel SSD management. As a middle ground between fully host-managed OCSSDs and traditional block-interface SSDs, Zoned Namespaces SSDs were subsequently introduced [4, 1]. Seshadri et al.

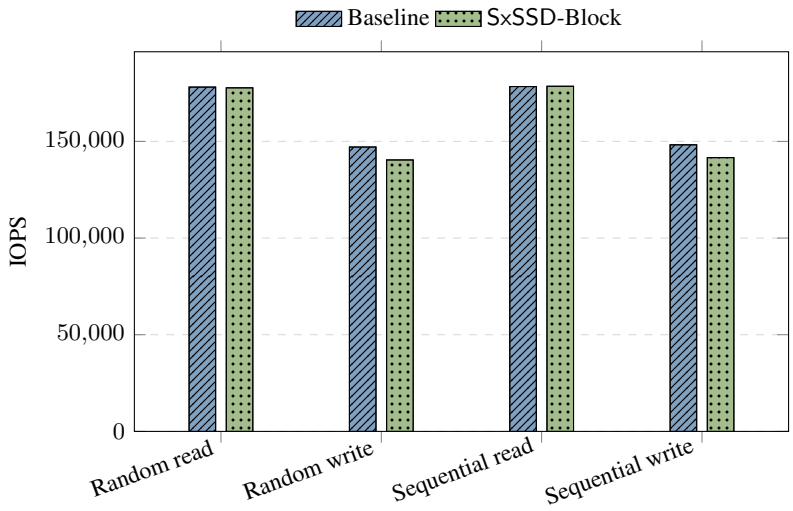

Figure 5: Peak random and sequential read/write IOPS under high-intensity 4 KiB fio workloads.

proposed Willow [59], an extensible SSD that enables device-enforced application-defined storage semantics through SSD Apps (e.g., Base-IO or Atomic-Writes). However, their design does not consider NAND flash and rather uses simulated phase change memory, which is not restricted by C1-C4. As a result, Willow adds new application-level operations to the SSD, rather than making the NAND flash FTL itself programmable. Additionally, Willow does not consider a compromised OS and therefore does not support security-critical storage semantics.

FTL Based Security Critical Storage Semantics. Previous work has demonstrated that several security critical applications can be supported by extending the FTL with special storage semantics. FlashGuard [19] and SSD-Insider [20] enable recovery from ransomware attacks by taking advantage of the out-of-place updates inherent in block-interface FTLs. PEARL enables plausible deniability by storing hidden bits in the same physical locations as public bits [21]. HiDCS modifies the FTL to enable self-auditing and self-repair for storage peers in the decentralized cloud, ensuring data integrity without relying on expensive consensus mechanisms [24].

Generic FTL Frameworks. HIL is a conceptual framework for FTL development that uses Logs as the fundamental building block [32]. It provides a flexible multi-layer mapping framework that also supports crash recovery. OX establishes a framework for designing custom FTLs on top of Open-Channel SSDs by deconstructing the FTL into modular components and defining their interactions [60].

9 Conclusion

In this work, we have introduced SxSSD, the first secure yet extensible software-defined SSD. SxSSD enables restricted access for trusted applications to program the FTL behavior by developing and installing custom policies. In addition, SxSSD is NAND flash aware and addresses the limitations of previous work by making the flash-to-OS interface changeable. Since the FTL is running on isolated flash memory hardware, it constitutes a critical security boundary. Thus, SxSSD supports the development and dynamic installation of security-critical storage semantics even when the operating system is compromised. To demonstrate this, we have implemented a case study in which we enable secure data recovery from ransomware attacks. In addition, we evaluate SxSSD across several real-world workloads.

Acknowledgment

This material is based upon work supported by the National Science Foundation Graduate Research Fellowship Program under Grant No. 2437847, and by the National Science Foundation under Grant Nos. 2225424, 2043022 and 2623031. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

References

[1] NVM Express, Inc. Nvm express zoned namespace command set specification. Technical Report Revision 1.3, NVM Express, Inc., March 2025. Ratified 2025-03-11.

[2] Open-channel solid state drives specification. `http://lightnvm.io/docs/OCSSD-2_0-20180129.pdf`, 2018. Revision 2.0, January 29, 2018. Accessed: 2026-01-05.

[3] Jian Ouyang, Shiding Lin, Song Jiang, Zhenyu Hou, Yong Wang, and Yuanzheng Wang. Sdf: software-defined flash for web-scale internet storage systems. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '14, pages 471–484, New York, NY, USA, 2014. Association for Computing Machinery.

[4] Matias Bjørling, Abutalib Aghayev, Hans Holmberg, Aravind Ramesh, Damien Le Moal, Gregory R. Ganger, and George Amvrosiadis. ZNS: Avoiding the block interface tax for flash-based SSDs. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 689–703. USENIX Association, July 2021.

[5] Theano Stavrinos, Daniel S. Berger, Ethan Katz-Bassett, and Wyatt Lloyd. Don't be a blockhead: zoned namespaces make work on conventional ssds obsolete. In *Proceedings of the Workshop on Hot Topics in Operating Systems*, HotOS '21, pages 144–151, New York, NY, USA, 2021. Association for Computing Machinery.

[6] Matias Bjørling, Javier Gonzalez, and Philippe Bonnet. LightNVM: The linux Open-Channel SSD subsystem. In *15th USENIX Conference on File and Storage Technologies (FAST 17)*, pages 359–374, Santa Clara, CA, February 2017. USENIX Association.

[7] Ivan Luiz Picoli, Niclas Hedam, Philippe Bonnet, and Pınar Tözün. Open-channel ssd (what is it good for). In *Conference on Innovative Data Systems Research*, 2020.

[8] Alberto Lerner and Philippe Bonnet. Not your grandpa's ssd: The era of co-designed storage devices. In *Proceedings of the 2021 International Conference on Management of Data*, SIGMOD '21, pages 2852–2858, New York, NY, USA, 2021. Association for Computing Machinery.

[9] Nitin Agrawal, Vijayan Prabhakaran, Ted Wobber, John D. Davis, Mark Manasse, and Rina Panigrahy. Design tradeoffs for ssd performance. In *USENIX 2008 Annual Technical Conference*, ATC'08, pages 57–70, USA, 2008. USENIX Association.

[10] Jun He, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. The unwritten contract of solid state drives. In *Proceedings of the Twelfth European Conference on Computer Systems*, EuroSys '17, pages 127–144, New York, NY, USA, 2017. Association for Computing Machinery.

[11] Jingpei Yang, Ned Plasson, Greg Gillis, Nisha Talagala, and Swaminathan Sundararaman. Don't stack your log on my log. In *2nd Workshop on Interactions of NVM/Flash with Operating Systems and Workloads (INFLOW 14)*, Broomfield, CO, October 2014. USENIX Association.

[12] Zhenhua Tan, Linbo Long, Jingcheng Shen, Renping Liu, Congming Gao, Kan Zhong, and Yi Jiang. Optimizing garbage collection for zns ssds via in-storage data migration and address remapping. *ACM Trans. Archit. Code Optim.*, 21(4), November 2024.

[13] Kyuhwa Han, Hyunho Gwak, Dongkun Shin, and Jooyoung Hwang. Zns+: Advanced zoned namespace interface for supporting in-storage zone compaction. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, pages 147–162. USENIX Association, July 2021.

[14] Ziyang Jiao, Janki Bhimani, and Bryan S. Kim. Wear leveling in ssds considered harmful. In *Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems*, HotStorage '22, pages 72–78, New York, NY, USA, 2022. Association for Computing Machinery.

[15] Feng Chen, Tian Luo, and Xiaodong Zhang. CAFTL: A Content-Aware flash translation layer enhancing the lifespan of flash memory based solid state drives. In *9th USENIX Conference on File and Storage Technologies (FAST 11)*, San Jose, CA, February 2011. USENIX Association.

[16] Jaeho Kim, Donghee Lee, and Sam H. Noh. Towards SLO complying SSDs through OPS isolation. In *13th USENIX Conference on File and Storage Technologies (FAST 15)*, pages 183–189, Santa Clara, CA, February 2015. USENIX Association.

[17] Jian Huang, Anirudh Badam, Laura Caulfield, Suman Nath, Sudipta Sengupta, Bikash Sharma, and Moinuddin K. Qureshi. FlashBlox: Achieving both performance isolation and uniform lifetime for virtualized SSDs. In *15th USENIX Conference on File and Storage Technologies (FAST 17)*, pages 375–390, Santa Clara, CA, February 2017. USENIX Association.

[18] Jaeyoung Do, Chen Luo, and David Lomet. Programming an ssd controller to support batched writes for variable-size pages. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*, pages 756–767, 2021.

[19] Jian Huang, Jun Xu, Xinyu Xing, Peng Liu, and Moinuddin K. Qureshi. Flashguard: Leveraging intrinsic flash properties to defend against encryption ransomware. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, CCS '17, pages 2231–2244, New York, NY, USA, 2017. Association for Computing Machinery.

[20] SungHa Baek, Youngdon Jung, Aziz Mohaisen, Sungjin Lee, and DaeHun Nyang. Ssd-insider: Internal defense of solid-state drive against ransomware with perfect data recovery. In *2018 IEEE 38th International Conference on Distributed Computing Systems (ICDCS)*, pages 875–884, 2018.

[21] Chen Chen, Anrin Chakraborti, and Radu Sion. PEARL: Plausibly deniable flash translation layer using WOM coding. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1109–1126. USENIX Association, August 2021.

[22] Weidong Zhu, Carson Stillman, Sara Rampazzi, and Kevin R. B. Butler. Enabling secure and efficient data loss prevention with a retention-aware versioning ssd. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, pages 171–185, New York, NY, USA, 2025. Association for Computing Machinery.

[23] Le Guan, Shijie Jia, Bo Chen, Fengwei Zhang, Bo Luo, Jingqiang Lin, Peng Liu, Xinyu Xing, and Luning Xia. Supporting transparent snapshot for bare-metal malware analysis on mobile devices. In *Proceedings of the 33rd annual computer security applications conference*, pages 339–349, 2017.

[24] Josh Dafoe, Niusen Chen, and Bo Chen. Hardware-assisted secure decentralized cloud storage via self-audit and self-repair. In *Proceedings of the 21st EAI International Conference on Security and Privacy in Communication Networks (SecureComm)*. EAI, 2025.

[25] Huaicheng Li, Mingzhe Hao, Michael Hao Tong, Swaminathan Sundararaman, Matias Bjørling, and Haryadi S. Gunawi. The CASE of FEMU: Cheap, accurate, scalable and extensible flash emulator. In *16th USENIX Conference on File and Storage Technologies (FAST 18)*, pages 83–90, Oakland, CA, February 2018. USENIX Association.

[26] Philippe Bonnet and Luc Bouganim. Flash device support for database management. In *CIDR 2011 - 5th Biennial Conference on Innovative Data Systems Research, Conference Proceedings*, pages 1–8, 01 2011.

[27] Trevor Pott and Iain Thomson. Supercapacitors have the power to save you from data loss. `https://www.theregister.com/2014/09/24/storage_supercapacitors/`, September 2014. Accessed: 2026-01-05.

[28] Rajat Kateja, Anirudh Badam, Sriram Govindan, Bikash Sharma, and Greg Ganger. Viyojit: Decoupling battery and dram capacities for battery-backed dram. In *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 613–626, 2017.

[29] Solid state drive market size, share & industry analysis, by interface type (serial ata (sata), serial attached scsi (sas), and others), by storage (under 500gb, 500gb – 1tb, 1tb – 2tb, and above 2tb), by end-user (individuals, enterprises, and others), and regional forecast, 2026 – 2034. `https://www.fortunebusinessinsights.com/solid-state-drive-market-109839`, 2026. Accessed: 2026-04-19.

[30] Renping Liu, Xiaoyan Xiao, Yang Zou, Peng Chen, Linbo Long, Anping Xiong, and Duo Liu. Zns-cleaner: Enhancing lifespan by reducing empty erase in zns ssds. *J. Syst. Archit.*, 157:103303, 2024.

[31] Li-Pin Chang. On efficient wear leveling for large-scale flash-memory storage systems. In *Proceedings of the 2007 ACM Symposium on Applied Computing*, SAC '07, pages 1126–1130, New York, NY, USA, 2007. Association for Computing Machinery.

[32] Jin-Yong Choi, Eyee Hyun Nam, Yoon Jae Seong, Jin Hyuk Yoon, Sookwan Lee, Hong Seok Kim, Jeongsu Park, Yeong-Jae Woo, Sheayun Lee, and Sang Lyul Min. Hil: A framework for compositional ftl development and provably-correct crash recovery. *ACM Trans. Storage*, 14(4), December 2018.

[33] Hong Seok Kim, Eyee Hyun Nam, Ji Hyuck Yun, Sheayun Lee, and Sang Lyul Min. P-bms: A bad block management scheme in parallelized flash memory storage devices. *ACM Trans. Embed. Comput. Syst.*, 16(5s), September 2017.

[34] Hans Holmberg. dm-zap: Host-based ftl for zns ssds. `https://github.com/westerndigitalcorporation/dm-zap`, 2021. Accessed: 2026-01-06.

[35] Western Digital Corporation. Kernel configuration. `https://zonedstorage.io/docs/linux/config`, 2025. Zoned Storage documentation. Accessed: 2026-01-06.

[36] Western Digital Corporation. Drive protection. Retrieved on January 01, 2026 from https://www.westerndigital.com/solutions/data-security/drive-protection.

[37] Micron Technology, Inc. Protecting your ssd and your data: Secure your ssd's firmware against ever-growing threats. https://datasheet.datasheetarchive.com/originals/crawler/micron.com/1c1a884377ab1954f2efc54b614636ec.pdf, 2016. Technical brief; accessed 2026-01-01.

[38] Jinghui Liao, Niusen Chen, Lichen Xia, Bo Chen, and Weisong Shi. Fspde: A full stack plausibly deniable encryption system for mobile devices. In *Proceedings of the Fourteenth ACM Conference on Data and Application Security and Privacy*, CODASPY '24, pages 265–276, New York, NY, USA, 2024. Association for Computing Machinery.

[39] Jinwoo Ahn, Junghee Lee, Yungwoo Ko, Donghyun Min, Jiyun Park, Sungyong Park, and Youngjae Kim. Diskshield: A data tamper-resistant storage for intel sgx. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, ASIA CCS '20, pages 799–812, New York, NY, USA, 2020. Association for Computing Machinery.

[40] Niusen Chen, Bo Chen, and Weisong Shi. A cross-layer plausibly deniable encryption system for mobile devices. In *Security and Privacy in Communication Networks*, 2022.

[41] Xiaohao Wang, Yifan Yuan, You Zhou, Chance C. Coats, and Jian Huang. Project almanac: A time-traveling solid-state drive. In *Proceedings of the Fourteenth EuroSys Conference 2019*, EuroSys '19, New York, NY, USA, 2019. Association for Computing Machinery.

[42] D. Dolev and A. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.

[43] Whitfield Diffie, Paul C. van Oorschot, and Michael J. Wiener. Authentication and authenticated key exchanges. *Designs, Codes and Cryptography*, 2:107–125, 1992.

[44] David McGrew. An interface and algorithms for authenticated encryption. RFC 5116, RFC Editor, January 2008.

[45] Aleph One. Smashing the stack for fun and profit. Phrack Magazine, Issue 49, 1996. Accessed: 2026-02-03.

[46] MITRE. CWE-416: Use After Free. https://cwe.mitre.org/data/definitions/416.html, 2024. Accessed: 2026-02-03.

[47] Al Bessey, Ken Block, Ben Chelf, Andy Chou, Bryan Fulton, Seth Hallem, Charles Henri-Gros, Asya Kamsky, Scott McPeak, and Dawson Engler. A few billion lines of code later: using static analysis to find bugs in the real world. *Commun. ACM*, 53(2):66–75, February 2010.

[48] Niclas Hedam. Delilah: Efficient ebpf offload for integrated data pipelines on computational storage, October 2024.

[49] OSDev Wiki. ATA Command Matrix. https://wiki.osdev.org/ATA_Command_Matrix, 2021. Accessed: 2026-04-29.

[50] INCITS T10 Technical Committee. SCSI Operation Codes: Numeric Sorted Listing. https://www.t10.org/lists/op-num.htm, 2015. Accessed: 2026-04-29.

[51] Josh Dafoe. SxSSD implementation and evaluation artifacts. https://github.com/jdafoe12/SxSSD/tree/3451afc74b4cd03063575c6070397759ecf78214, 2026. Commit 3451afc74b4cd03063575c6070397759ecf78214.

[52] Chunghan Lee, Tatsuo Kumano, Tatsuma Matsuki, Hiroshi Endo, Naoto Fukumoto, and Mariko Sugawara. Understanding storage traffic characteristics on enterprise virtual desktop infrastructure. In *Proceedings of the 10th ACM International Systems and Storage Conference*, SYSTOR '17, New York, NY, USA, 2017. Association for Computing Machinery.

[53] Dushyanth Narayanan, Austin Donnelly, and Antony Rowstron. Write Off-Loading: Practical power management for enterprise storage. In *6th USENIX Conference on File and Storage Technologies (FAST 08)*, San Jose, CA, February 2008. USENIX Association.

[54] Ricardo Koller and Raju Rangaswami. I/o deduplication: Utilizing content similarity to improve i/o performance. *ACM Trans. Storage*, 6(3), September 2010.

[55] Alibaba Group. Alibaba Cloud Block Trace Dataset. https://github.com/alibaba/block-traces, 2020. Block I/O traces from 1000 virtual disks collected in a production cluster, accessed 2026-04-24.

[56] MoatLab. FEMU Best Practice. https://github.com/MoatLab/FEMU/wiki/FEMU-Best-Practice, 2020. Accessed: 2026-04-24.

[57] Jens Axboe. fio: Flexible i/o tester. `https://github.com/axboe/fio`, 2021. GitHub repository, accessed 2026-04-24.

[58] Jianquan Zhang, Dan Feng, Jianlin Gao, Wei Tong, Jingning Liu, Yu Hua, Yang Gao, Caihua Fang, Wen Xia, Feiling Fu, and Yaqing Li. Application-aware and software-defined ssd scheme for tencent large-scale storage system. In *2016 IEEE 22nd International Conference on Parallel and Distributed Systems (ICPADS)*, pages 482–490, 2016.

[59] Sudharsan Seshadri, Mark Gahagan, Sundaram Bhaskaran, Trevor Bunker, Arup De, Yanqin Jin, Yang Liu, and Steven Swanson. Willow: A User-Programmable SSD. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 67–80, Broomfield, CO, October 2014. USENIX Association.

[60] Ivan Luiz Picoli. *OX: Deconstructing the FTL for Computational Storage*. PhD thesis, IT University of Copenhagen, July 2019.